OPEN SOURCE · MIT · VERSION 2.0

ng-checkup.

A one-minute modernization report for any Angular codebase: how far it is from current Angular, what breaks on the next upgrade, and where the work is. Every finding names the file and the line.

```
$ npx ng-checkup-cli
```

14
areas checked, 12 of them graded

0
things to install or configure first

19
public Angular repositories it is tested against

6
places it runs: CLI, CI, editor, agents, fleets, PRs

```
ng-checkup 2.0.0  ang2-conduit · Angular CLI · Angular 12.2.17
71 TypeScript files, 19 templates, 1 483 lines of application code, analysed in 270 ms

[POOR]  Angular version & tooling             Angular 12.2.17, 10 majors behind 22.
[GOOD]  Deprecated APIs                       Nothing deprecated as of Angular 12 is in use.
[POOR]  Change detection & zoneless readiness Runs on zone.js; 0% of 18 components use OnPush.
[POOR]  Components & templates                18 components: 0% standalone, 0% signal inputs.
[FAIR]  State management                      No classic NgRx, 0 signal()/computed() calls.
[GOOD]  Architecture                          79% of 14 routes lazy, 0 files over 500 lines.
[POOR]  Testing                               Karma, 0 spec files.
[POOR]  Type safety                           strictNullChecks off, strictTemplates off, 13 explicit any.
[GOOD]  Security                              No sanitizer bypasses, direct DOM HTML writes or eval.
[INFO]  Accessibility                         7 accessibility defects in 6 templates.
[GOOD]  Circular dependencies                 No import cycles between application files.

Score 56/100

Top priorities
  1. high   Angular 12 is 10 majors behind 22 and out of support
  2. high   Unit tests run on Karma, which is deprecated
```

The public RealWorld example app, as the tool reports it today

github.com/stefankuburovic/ng-checkup

HOW IT WORKS

One read of the repository, pure checks, any format

Nothing is built, served or executed. The tool reads the source as text and syntax trees, so it runs on a repository it has never seen, on any Angular version from 2 to 22, and on AngularJS on its way there.

1. Read

One walk over the folder: TypeScript, templates, stylesheets, translation bundles, `angular.json` or Nx `project.json`, the lockfile, build stats.

→

2. Facts

Each file parsed once with the TypeScript compiler: imports, components, signals, subscriptions, tests, keys, colours. Cached per file in watch mode.

→

3. Checks

Fourteen pure functions of those facts. No check touches the disk, which is what keeps every result reproducible.

→

4. Report

Terminal, HTML, Markdown, JSON, SARIF, GitLab Code Quality, a pull-request comment, or a client report.

AREA	WHAT IT READS	COUNTS
Angular version	Installed and declared version, majors behind, end of support, tooling	GRADED
Deprecated APIs	Every imported Angular, NgRx and RxJS symbol against the `@deprecated` tags of the exact version, and what the next major removes	GRADED
Third-party compatibility	Which packages block the upgrade, or do not accept the installed Angular	GRADED
Change detection	zone.js, zoneless readiness, OnPush share, NgZone and zone-based test helpers	GRADED
Components & templates	Standalone, signal inputs and outputs, built-in control flow, `@defer`, subscriptions nothing ends, colours outside the tokens	GRADED
State management	Classic NgRx, signal store, signals, `BehaviorSubject` services	GRADED
Architecture	Lazy routes, large files, constructor injection, unused files, components and pipes, git hotspots	GRADED
Testing	Runner, spec coverage by file, tests that assert nothing, unwaited assertions, `NO_ERRORS_SCHEMA`	GRADED
Type safety	`strictNullChecks`, `strictTemplates`, `noImplicitAny`, `any`, `x!`, `@ts-ignore`	GRADED
Security	Known vulnerabilities (osv.dev), sanitizer bypasses, HTML written into the DOM, `eval`	GRADED
Circular dependencies	Import cycles through relative paths, barrels, `paths` and `baseUrl`	GRADED
Bundle size	Initial bundle of the last `ng build --stats-json` against its budget	GRADED
Translations	ngx-translate and Transloco keys that resolve to no string in some language	LISTED
Accessibility	Unlabelled controls, unnamed buttons, mouse-only `(click)`, images without `alt`	LISTED
AngularJS / Primo NDE	Legacy code and its migration plan, only when there is any	WHEN PRESENT

Graded areas make the score: good counts 100, fair 60, poor 20. **Listed** areas report findings and baseline measures without a grade, so a new defect still fails CI while the score stays comparable between releases.

WHERE IT RUNS

From one command to every pull request

The same analysis behind every entry point. Pick the one that fits the moment: a first look, a guard in CI, a hint while editing, a question from an agent.

FIRST LOOK

The command line

Any folder, or a remote repository it clones shallow and deletes again. Reports as a self-contained page, Markdown or JSON.

```
npx ng-checkup-cli github:owner/repo --
report checkup.html
```

ADOPT IT

Setup and baseline

Adds a script, a config, a baseline of today's numbers and a CI step. CI then fails only when a number gets worse, never on old debt.

```
ng add ng-checkup-cli
npx ng-checkup-cli --fail-on regression
```

CONTINUOUS INTEGRATION

Every pull request

GitHub Action with a sticky comment and SARIF, GitLab component with Code Quality, and generated pipelines for Azure DevOps and Bitbucket.

```
uses: stefankuburovic/ng-checkup@v2
component: …/ng-checkup/ng-checkup@2
```

ONLY WHAT CHANGED

Findings of a branch

Keeps the findings in files the branch touched, so a review sees what this change adds. Grades and the baseline stay the whole repository's.

```
npx ng-checkup-cli --since origin/main
```

WHILE EDITING

VS Code and Cursor

Findings in the Problems panel at their line, the reason on hover, the score in the status bar. Re-parses only the saved file.

```
Marketplace · Open VSX: ng-checkup
```

FOR AGENTS

MCP server

Claude Code, Cursor or Copilot ask directly: the report, the findings of one file, the upgrade plan. Offline, no extra dependency.

```
claude mcp add ng-checkup -- npx -y ng-
checkup-cli mcp
```

MANY REPOSITORIES

Fleet page

One ranked page across every front end a company or an agency runs, worst first, with a grade per area for each.

```
npx ng-checkup-cli fleet repos.txt --
report fleet.html
```

PLAN AND HISTORY

Upgrade plan, trend, watch

A step per major with what to fix first, the last twelve months from git as charts, and a live re-run on every save.

```
npx ng-checkup-cli --plan --report
plan.md
npx ng-checkup-cli history
```

THE PACKAGE FAMILY

What ships, and what leaves your machine

PART	WHERE	WHAT IT IS
ng-checkup-cli	npm	The CLI, the programmatic API (`runCheckup` , renderers) and the MCP server. Installs the `ng-checkup` command.
GitHub Action	GitHub Marketplace	`stefankuburovic/ng-checkup@v2` : pull-request comment, job summary, SARIF for code scanning, reports as artifacts.
GitLab component	GitLab CI/CD Catalog	`ng-checkup@2` : merge request Code Quality widget, artifacts, one comment per merge request. A template for self-managed GitLab.
Editor extension	VS Code Marketplace, Open VSX	Inline diagnostics, hover detail, status-bar score and the HTML report one click away.

Built to be trusted with a codebase

- **Reads, never runs.** Nothing from the repository is installed, built or executed, a cloned one included.
- **Offline when asked.** `--offline` sends nothing. Otherwise only package names and versions go to the npm registry and osv.dev; no source code ever leaves the machine.
- **Tested on real code.** Every rule is checked against 19 public Angular repositories at pinned commits, and any number a change moves is reviewed before release.
- **Errs towards silence.** A rule that cannot be sure stays quiet, and the score only counts what is graded.

New in 2.0

- Translations and accessibility, listed without a grade
- Test quality: tests that cannot fail
- Subscriptions nothing ends, colours outside the tokens
- `--since` for the findings of one branch
- MCP server for editor agents
- Fleet page across many repositories
- Azure DevOps and Bitbucket pipelines in `init`

Get started in three steps

1 · LOOK

Run it once, read the score and the top five priorities.

```
npx ng-checkup-cli --report checkup.html
```

2 · LOCK IN

Record today's numbers and add the CI step for the CI you use.

```
npx ng-checkup-cli init
```

3 · IMPROVE

Work down the plan; every number that drops is locked in.

```
npx ng-checkup-cli baseline
```

Want someone to act on the report?

I run a free Angular Health Check on your codebase: the full report, a walkthrough of what matters first, and an upgrade plan with an estimate per step. The modernization work itself, from zoneless to the next major, is what I do for a living.

stefankuburovic.github.io · **linkedin.com/in/stefankuburovic**
github.com/stefankuburovic/ng-checkup